



Topology & Entity Context

Series: OPLOGS | **Notebook:** 6 of 8 | **Created:** December 2025

Leveraging Entity Relationships in Log Analysis

This notebook explores how Dynatrace enriches logs with entity context (hosts, processes, services, Kubernetes) for topology-aware analysis.

Table of Contents

1. Entity Types Overview
2. Host Topology
3. Process Group Topology
4. Kubernetes Topology
5. Service Mapping
6. Cross-Entity Correlation
7. Using Entity IDs for Lookups
8. Topology-Based Alerting Patterns

Prerequisites

-  Access to a Dynatrace environment with log data
-  Completed OPLOGS-01 through OPLOGS-05
-  Understanding of Dynatrace entity model (helpful)

1. Entity Types Overview

Dynatrace automatically enriches logs with entity context:



Entity Field	Description	Example
dt.entity.host	Host entity ID	HOST-ABC123
dt.entity.process_group	Process group ID	PROCESS_GROUP-XYZ789
dt.entity.process_group_instance	PGI ID	PROCESS_GROUP_INSTANCE-DEF456
dt.entity.service	Service entity ID	SERVICE-QRS012
dt.entity.kubernetes_cluster	K8s cluster ID	KUBERNETES_CLUSTER-TUV345

Kubernetes Context Fields

Field	Description
k8s.namespace.name	Kubernetes namespace
k8s.pod.name	Pod name
k8s.pod.uid	Pod unique identifier
k8s.container.name	Container name
k8s.cluster.name	Cluster name
k8s.deployment.name	Deployment name
k8s.workload.name	Workload name
k8s.workload.kind	Workload type (Deployment, StatefulSet, etc.)

```
// Discover available entity types in your logs
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    with_host = countIf(isNotNull(dt.entity.host)),
    with_process_group = countIf(isNotNull(dt.entity.process_group)),
    with_service = countIf(isNotNull(dt.entity.service)),
    with_k8s_cluster = countIf(isNotNull(dt.entity.kubernetes_cluster)),
    with_k8s_namespace = countIf(isNotNull(k8s.namespace.name))
}
```

2. Host Topology

Analyze logs by host to understand infrastructure patterns.

```
// Log volume by host
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.host)
| summarize {log_count = count()}, by: {dt.entity.host}
| sort log_count desc
| limit 15
```

```
// Error distribution by host
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.host)
| summarize {
    total = count(),
    errors = countIf(loglevel == "ERROR" OR loglevel == "SEVERE")
}, by: {dt.entity.host}
| fieldsAdd error_rate = (errors * 100.0) / total
| sort errors desc
| limit 15
```

```
// Host with log.source breakdown
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.host)
| summarize {count = count()}, by: {dt.entity.host, log.source}
| sort count desc
| limit 20
```

3. Process Group Topology

Process groups represent logical application components across hosts.

```
// Logs by process group
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.process_group)
| summarize {log_count = count()}, by: {dt.entity.process_group}
| sort log_count desc
| limit 15
```

```
// Process group error analysis
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.process_group)
| filter loglevel == "ERROR"
| fieldsAdd content_preview = substring(content, from: 0, to: 80)
| summarize {error_count = count()}, by: {dt.entity.process_group,
content_preview}
| sort error_count desc
| limit 20
```

```
// Process group to host mapping
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.process_group) AND isNotNull(dt.entity.host)
| summarize {count = count()}, by: {dt.entity.process_group, dt.entity.host}
| sort count desc
| limit 20
```

4. Kubernetes Topology

OpenPipeline enriches container logs with rich Kubernetes context.

```
// Logs by Kubernetes namespace
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize {log_count = count()}, by: {k8s.namespace.name}
| sort log_count desc
| limit 15
```

```
// Kubernetes namespace with error breakdown
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize {
    total = count(),
    errors = countIf(loglevel == "ERROR" OR loglevel == "WARN")
}, by: {k8s.namespace.name}
| fieldsAdd error_percentage = round((errors * 100.0) / total, decimals: 2)
| sort errors desc
| limit 10
```

```
// Pod-level analysis
fetch logs, from: now() - 1h
| filter isNotNull(k8s.pod.name)
| summarize {
    log_count = count(),
    error_count = countIf(loglevel == "ERROR")
}, by: {k8s.namespace.name, k8s.pod.name}
| sort error_count desc
| limit 20
```

```
// Workload analysis (Deployments, StatefulSets, etc.)
fetch logs, from: now() - 1h
| filter isNotNull(k8s.workload.name)
| summarize {
    log_count = count(),
    unique_pods = countDistinct(k8s.pod.name)
}, by: {k8s.namespace.name, k8s.workload.kind, k8s.workload.name}
| sort log_count desc
| limit 15
```

```
// Container-level detail
fetch logs, from: now() - 1h
| filter isNotNull(k8s.container.name)
| summarize {log_count = count()}, by: {k8s.namespace.name, k8s.pod.name,
k8s.container.name}
| sort log_count desc
| limit 20
```

5. Service Mapping

Connect logs to Dynatrace-detected services for full observability.

```
// Logs by service entity
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.service)
| summarize {log_count = count()}, by: {dt.entity.service}
| sort log_count desc
| limit 15
```

```
// Service error rates from logs
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.service)
| summarize {
    total = count(),
    errors = countIf(loglevel == "ERROR")
}, by: {dt.entity.service}
| fieldsAdd error_rate = round((errors * 100.0) / total, decimals: 2)
| sort error_rate desc
| limit 15
```

```
// Service to process group relationship
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.service) AND isNotNull(dt.entity.process_group)
| summarize {count = count()}, by: {dt.entity.service,
dt.entity.process_group}
| sort count desc
| limit 20
```

6. Cross-Entity Correlation

Use entity context to correlate logs across the topology.

```
// Full topology view: Cluster > Namespace > Pod > Container
fetch logs, from: now() - 1h
| filter isNotNull(k8s.cluster.name)
| summarize {log_count = count()}, by: {
    k8s.cluster.name,
    k8s.namespace.name,
    k8s.workload.name,
    k8s.pod.name
}
| sort log_count desc
| limit 25
```

```
// Entity coverage report
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    host_coverage = round((countIf(isNotNull(dt.entity.host)) * 100.0) /
count(), decimals: 1),
    pg_coverage = round((countIf(isNotNull(dt.entity.process_group)) * 100.0)
/ count(), decimals: 1),
    service_coverage = round((countIf(isNotNull(dt.entity.service)) * 100.0)
/ count(), decimals: 1),
    k8s_coverage = round((countIf(isNotNull(k8s.namespace.name)) * 100.0) /
count(), decimals: 1)
}
```

```
// Logs without entity context (potential configuration issue)
fetch logs, from: now() - 1h
| filter isNull(dt.entity.host) AND isNull(dt.entity.process_group)
| summarize {orphan_count = count()}, by: {dt.openpipeline.source}
| sort orphan_count desc
```

```
// Trace correlation: Logs with trace context
fetch logs, from: now() - 1h
| filter isNotNull(trace_id) OR isNotNull(span_id)
| summarize {
    logs_with_trace = count(),
    unique_traces = countDistinct(trace_id)
}, by: {k8s.namespace.name}
| sort logs_with_trace desc
| limit 10
```

7. Using Entity IDs for Lookups

Entity IDs enable cross-data-type correlation.

```
// Get distinct entity IDs for a namespace
fetch logs, from: now() - 1h
| filter k8s.namespace.name == "hipstershop"
| summarize {
    unique_hosts = collectDistinct(dt.entity.host),
    unique_pgs = collectDistinct(dt.entity.process_group),
    unique_services = collectDistinct(dt.entity.service)
}
```

```
// Find logs for a specific entity (replace with actual entity ID)
// fetch logs, from: now() - 1h
// | filter dt.entity.host == "HOST-XXXXXX"
// | summarize {count = count()}, by: {loglevel}

// Discovery query to find entity IDs
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.host)
| summarize {sample = takeFirst(dt.entity.host)}, by: {k8s.namespace.name}
| limit 10
```

8. Topology-Based Alerting Patterns

Use entity context to create meaningful alert conditions.

```
// Alert pattern: Errors per namespace (for threshold alerting)
fetch logs, from: now() - 15m
| filter loglevel == "ERROR"
| summarize {error_count = count()}, by: {k8s.namespace.name}
| filter error_count > 10
| sort error_count desc
```

```
// Alert pattern: Hosts with high error rate
fetch logs, from: now() - 15m
| filter isNotNull(dt.entity.host)
| summarize {
    total = count(),
    errors = countIf(loglevel == "ERROR")
}, by: {dt.entity.host}
| filter total > 100 // Minimum sample size
| fieldsAdd error_rate = (errors * 100.0) / total
| filter error_rate > 5 // Alert if >5% errors
| sort error_rate desc
```

```
// Alert pattern: Pod restarts (look for startup patterns)
fetch logs, from: now() - 1h
| filter contains(content, "started") OR contains(content, "initializing")
| filter isNotNull(k8s.pod.name)
| summarize {startup_count = count()}, by: {k8s.namespace.name, k8s.pod.name}
| filter startup_count > 3 // More than 3 starts in 1h = potential crash
loop
| sort startup_count desc
```




Summary

In this notebook, you learned:

- ✓ **Entity types** - HOST, PROCESS_GROUP, SERVICE, KUBERNETES_CLUSTER
 - ✓ **Host topology** - Log volume and errors by host
 - ✓ **Process groups** - Application component analysis
 - ✓ **Kubernetes context** - Namespace, pod, workload, container
 - ✓ **Service mapping** - Connecting logs to detected services
 - ✓ **Cross-entity correlation** - Full topology views
 - ✓ **Alerting patterns** - Threshold-based topology alerts
-



Next Steps

Continue to **OPLOGS-07: Analytics & Dashboards** for aggregation and visualization patterns.



References

- [Dynatrace Entity Model](#)
 - [Kubernetes Monitoring](#)
 - [Log Enrichment](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.